

Einsums

Suhrid Shakhar Ghosh

July 2025

Outline

- Introduction to Einsums
- Matrix Multiplication as Einsums
- Scalar Vector Multiplication as Einsums
- Convert Einsum to CUDA Kernel with Tiling
- More example usage of Einsums

Introduction to Einsums

Einsum is short for Einstein summation convention

1-D Matrix Summation

$$A = [1, 2, 3, 4, \dots, n]$$

$$Sum = \sum_i A_i$$

2-D Matrix Summation

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$Sum = \sum_{ij} B_{ij}$$

Introduction to Einsums (continued)

1-D Matrix Summation

- Has only 1 axis (rank)
- Maps to a single scalar value (Dimension 0)
- $i \rightarrow$

Introduction to Einsums (continued)

2-D Matrix Summation

- Has 2 axes (ranks)
- Maps to a single scalar value (Dimension 0)
- " $ij \rightarrow$ "

Matrix Multiplication as Einsum

Given two matrices A and B of Dimension $m \times n$ and $n \times k$.

Results in a $m \times k$ matrix C .

$$\begin{aligned} & \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \cdot \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} \\ = & \begin{bmatrix} (1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11) & (1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12) \\ (4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11) & (4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12) \end{bmatrix} \\ = & \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix} \end{aligned}$$

Matrix Multiplication as Einsum (continued)

Einsum Representation

```
einsum("ij,jk->ik", A, B)
```

Matrix Multiplication as Einsum (continued)

How einsum works?

- The $\rightarrow$ symbol denotes transform (output arrow)
- Anything on left of the symbol defines inputs (separated by comma $(,)$)
- The letters on the right of the arrow symbol are outputs
- Each letter on both input and output denotes ranks of respective matrix

Matrix Multiplication as Einsum (continued)

Example

- Notation: $ij,jk \rightarrow ik$
- 1st input has dimension $i \times j$, 2nd input has dimension $j \times k$
- Output has dimension $i \times k$
- The dimension j is summed over

Matrix Multiplication as Einsum (continued)

How to translate?

- Write nested for loops for the ranks that appear on the output (outer loops)
- Initialize a sum variable to 0
- Write nested for loops for the ranks that needs to be sum over (inner loops)
- Write to output for each outer loop items

Matrix Multiplication as Einsum (continued)

Nested Loop Code for Matrix Multiplication

```
for i in range(2):  
    for j in range(3):  
        total = 0  
        for k in range(2):  
            total += A[i, k] * B[k, j]  
        result[i, j] = total
```

This approach works for any einsum!

Scalar Vector Multiplication as Einsums

Vector A (Dimension 1), Scalar B (Dimension 0)

```
einsum("i,j->i", A, B)
```

Convert Einsums to CUDA Kernel with Tiling

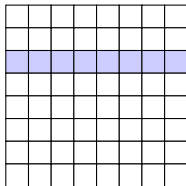
Which rank does threads represent?

The ranks that appear on the output can be calculated independently on separate threads

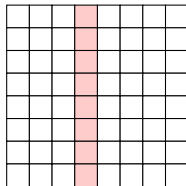
Whom to tile?

The item that are summed over are candidate for tiling, i.e., those ranks that appear in input but not in output are tiling eligible.

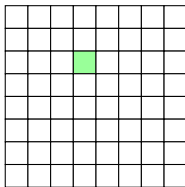
Convert Einsums to CUDA Kernel with Tiling (continued)



Matrix A



Matrix B



Matrix C ($A \times B$)

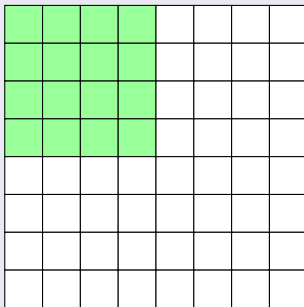
Convert Einsums to CUDA Kernel with Tiling (continued)

$ij,jk \rightarrow ik$

- We can process i and k rank in each thread
- We can tile j rank to get acceleration

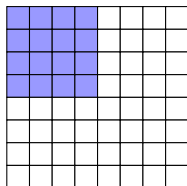
Convert Einsums to CUDA Kernel with Tiling (continued)

Assume launching block of size 4×4

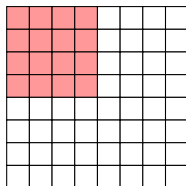


Matrix C ($A \times B$)

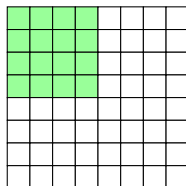
Convert Einsums to CUDA Kernel with Tiling (continued)



Matrix A

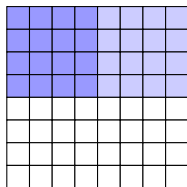


Matrix B

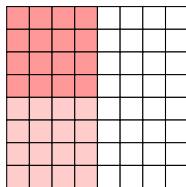


$$\text{Matrix C} = \text{A} \times \text{B}$$

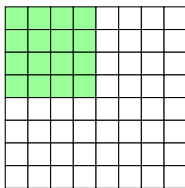
Convert Einsums to CUDA Kernel with Tiling (continued)



Matrix A



Matrix B



$$\text{Matrix C} = \text{A} \times \text{B}$$

Convert Einsums to CUDA Kernel with Tiling (continued)

Parallel thread index

```
unsigned int i = blockIdx.x * TILE_SIZE + threadIdx.x; // RANK i
unsigned int k = blockIdx.y * TILE_SIZE + threadIdx.y; // RANK k
```

Tiling

```
for (int t = 0; t < (N + TILE_SIZE - 1) / TILE_SIZE; ++t) {
    // Load A tile
    // Load B tile
    // Synchronize threads
    // Perform the multiplication and summation
}
```

More example usage of Einsums

Row sum

```
einsum('ij->i', A)
```

Column sum

```
einsum('ij->j', A)
```

More example usage of Einsums (Continued)

Element wise multiplication

```
einsum('ij,ij->ij', A, B)
```

Outer product

```
einsum('i,j->ij', A, B)
```

More example usage of Einsums (Continued)

Batch matrix multiplication

```
einsum('ijk,ikl->ikl', A, B)
```

Diagonal elements of a matrix

```
einsum('ii->i', A)
```